Univerzitet u Novom Sadu
Fakultet tehničkih nauka
Primenjeno softversko inženjerstvo

ELEMENTI RAZVOJA SOFTVERA

Vežba 1

Zahtevi, backlog, Git i timski razvoj

Radni materijal za računarske vežbe, samostalno ponavljanje i projektni rad.

Akademska 2026/2027. godina
Dopunjena Vežba 1: korisničke priče i simulirani sprintovi u PYXIS AI.

Sadržaj

Vežba 1 - Zahtevi, backlog, Git i timski razvoj 3

1.1. User Story i čeklista 3

1.2. Backlog, Tapiz Boards i plan rada 4

1.3. Git kao sledljiv zapis razvoja 12

1.4. Pull request, pregled koda i konflikti 13

Vežba 1 - Zahtevi, backlog, Git i timski razvoj

Razvoj softvera počinje razumevanjem problema, ali se kvalitet tog razumevanja potvrđuje tek kada zahtev postane proverljiva promena u proizvodu. U ovoj vežbi zahtevi, planiranje rada i Git čine jedan razvojni tok: poslovna potreba se zapisuje kao korisnička priča u jednoj rečenici, rad se razlaže kroz čeklistu, promena se implementira na izdvojenoj grani, pregleda kroz pull request i integriše nakon provere.

Od zahteva do integrisane promene

01	**Potreba** problem ili korisnička vrednost
02	**Backlog** korisnička priča i čeklista
03	**Grana** izolovana implementacija
04	**Pull request** pregled, testovi i diskusija
05	**Integracija** proverena promena u glavnoj grani

Slika 1.1 - Razvojni trag treba da omogući timu da poveže zahtev, odluku, implementaciju i proveru rezultata.

> **Osnovna ideja**
> Git istorija nije rezervna kopija završnog rešenja. Backlog nije spisak želja. Pull request nije formalnost. Svaki od ovih elemenata predstavlja deo proverljivog procesa razvoja.

1.1. User Story i čeklista

User Story je korisnička priča koja u jednoj rečenici opisuje ulogu korisnika, radnju ili mogućnost koju želi i cilj koji time postiže. U polje Opis studenti unose samo tu jednu rečenicu po sledećem šablonu.

> **Šablon korisničke priče**
> Kao [uloga], želim/hoću/mogu [da uradim nešto], kako bih [postigao cilj ili korist].

Deo rečenice	Šta opisuje	Primer
Kao [uloga]	Ko koristi funkcionalnost?	Kao nastavnik laboratorije
Želim/hoću/mogu [radnju]	Šta korisnik želi ili može da uradi?	želim da rezervišem slobodnu opremu za određeni termin
Kako bih [cilj ili korist]	Šta korisnik time postiže?	kako bih obezbedio opremu za izvođenje vežbe

Tabela 1.1.1

> **Primer jedne korisničke priče**
> Kao nastavnik laboratorije, želim da rezervišem slobodnu opremu za određeni termin, kako bih obezbedio opremu za izvođenje vežbe.

Čeklista se vodi u zasebnom delu kartice. Ona razlaže realizaciju priče na konkretne korake koje tim označava nakon izvršenja. Rečenica opisuje korisničku potrebu, a čeklista organizuje rad na njenoj realizaciji.

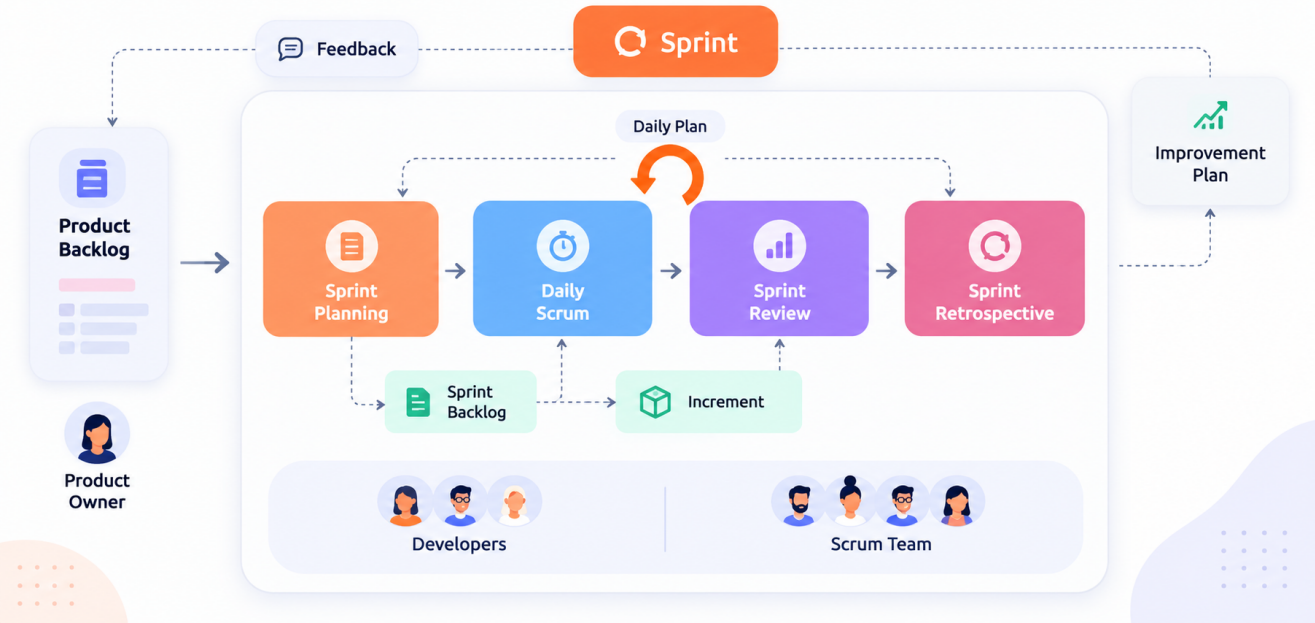
Primer čekliste za ovu priču

- Pripremiti prikaz slobodne opreme i izbor termina.
- Implementirati kreiranje rezervacije.
- Proveriti da se rezervacija ispravno čuva i prikazuje.
- Pregledati kod i integrisati promenu.
- Pripremiti kratku demonstraciju funkcionalnosti.

> **Rad na vežbi**
> Za jednu funkcionalnost svog projekta napisati tačno jednu rečenicu po šablonu „Kao [uloga], želim/hoću/mogu [radnju], kako bih [postigao cilj]“. Zatim u zasebnoj čeklisti navesti korake realizacije. Proveriti da se iz rečenice jasno vide uloga, radnja i korist, a da svaka stavka čekliste predstavlja konkretan korak rada.

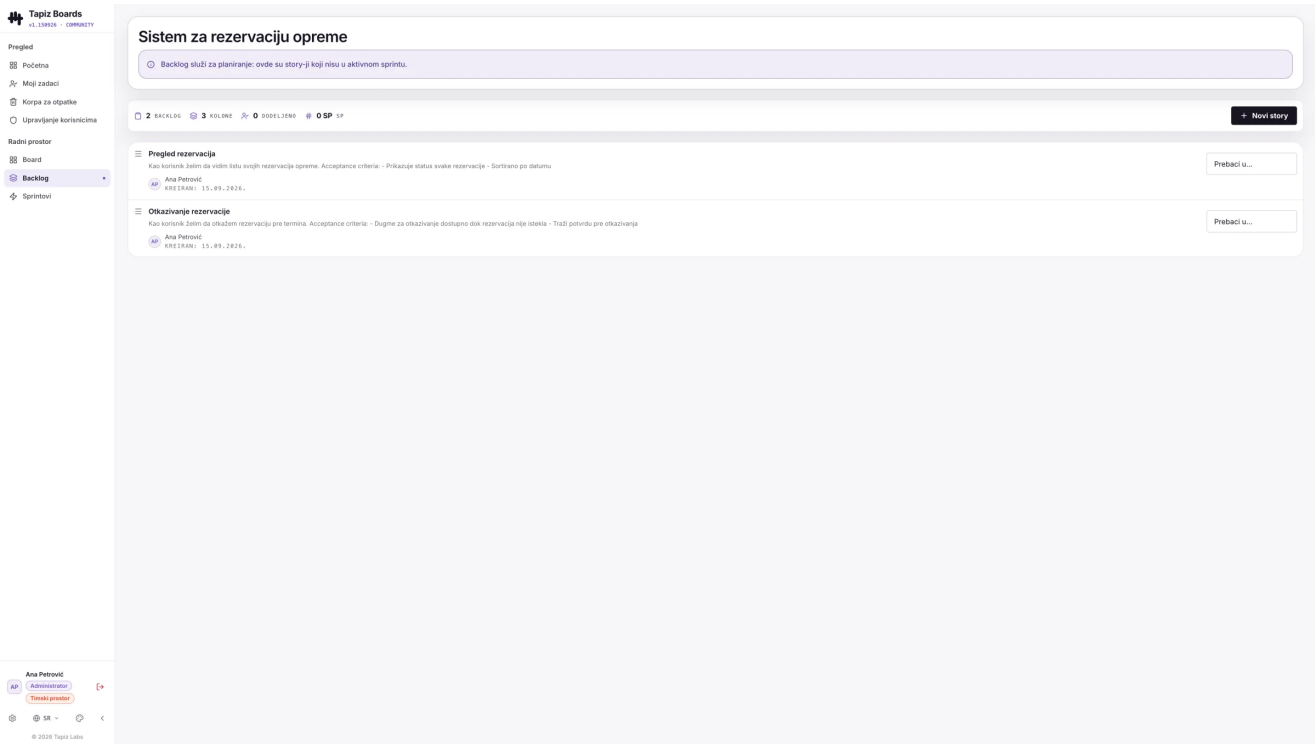
1.2. Backlog, Tapiz Boards i plan rada

Rad tima se odvija u ponovljivim ciklusima (sprintovima) unutar kojih se planira, implementira i preispituje ograničen obim posla. Product Backlog je zajednički izvor svih budućih stavki; Sprint Backlog i Increment nastaju kada tim za tekući sprint izabere i realizuje deo tog posla.



Slika 1.2.1 - Product Backlog napaja Sprint Planning; tim potom prolazi kroz Daily Scrum, Sprint Review i Sprint Retrospective, dok povratne informacije i plan unapređenja zatvaraju ciklus.

Backlog je uređena lista rada. Svaka stavka sadrži korisničku priču zapisanu u jednoj rečenici i zasebnu čeklistu koraka. Prioritet, odgovorna osoba, procena i sprint popunjavaju se u predviđenim poljima kartice. Tako tim vidi šta korisnik želi, zašto mu je to potrebno i kako će organizovati rad.

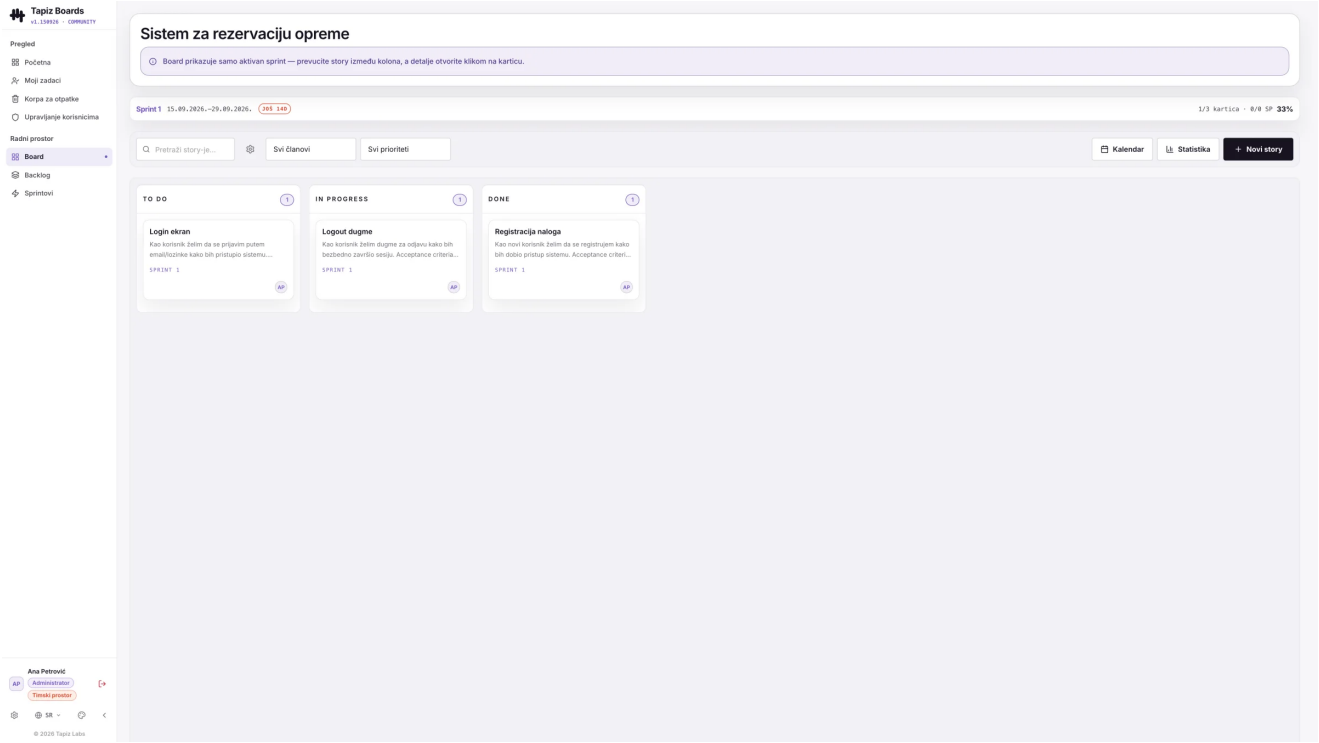


Slika 1.2.2 - Product Backlog u Tapiz Boards-u omogućava timu da sagleda prioritete, stanje pripreme i plan narednog rada.

Status	Značenje
Backlog	Stavka postoji, ali još nije spremna za neposredan rad.

Status	Značenje
Ready	Rečenica jasno opisuje ulogu, radnju i korist, a čeklista određuje korake rada.
In Progress	Implementacija je u toku i postoji odgovorna osoba ili par.
Code Review	Promena je implementirana i čeka stručni pregled koda.
QA/Verify	Proverava se da funkcionalnost omogućava korisniku radnju i cilj iz njegove priče.
Done	Promena je integrisana i ispunjava dogovorenu Definition of Done.

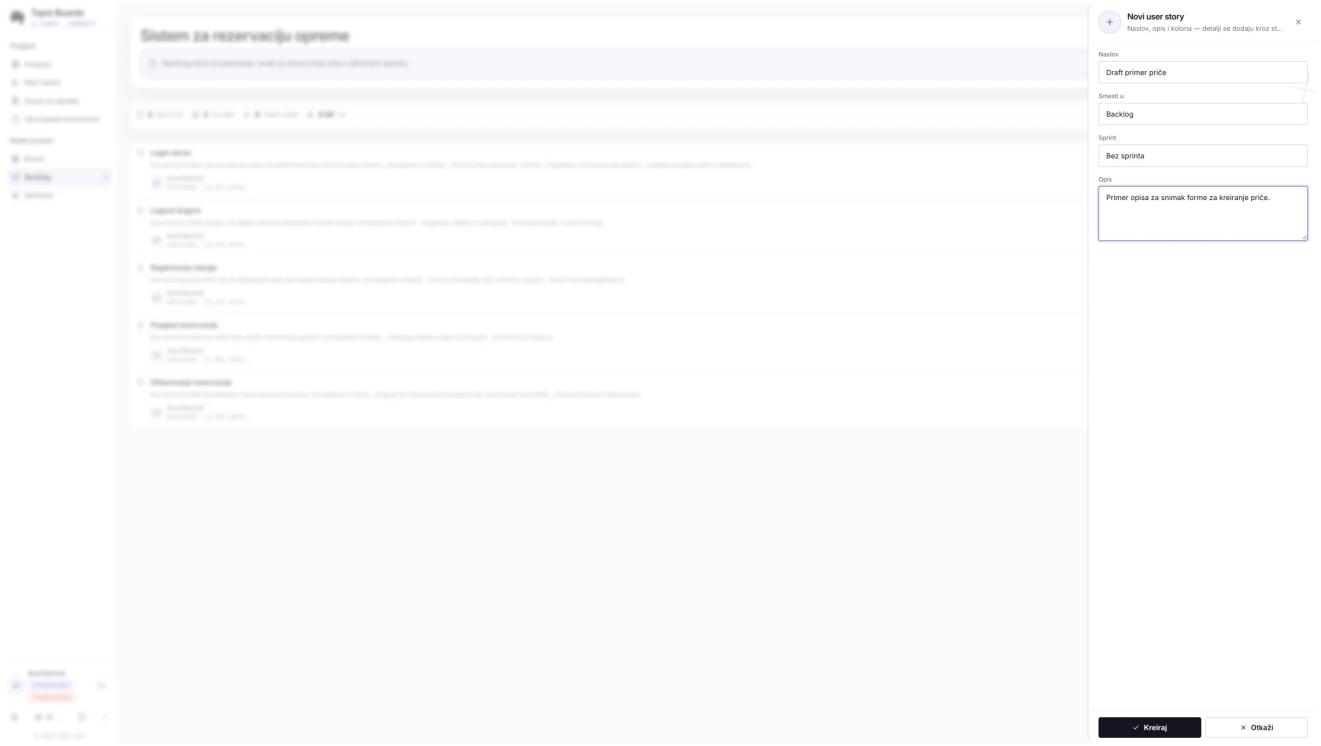
Tabela 1.2.1



Slika 1.2.3 - Radna tabla prikazuje stvarno stanje stavki u dogovorenom razvojnom procesu.

> **Status mora odgovarati stvarnom stanju**
> Status Done znači da je realizovana funkcionalnost iz korisničke priče, da su završeni koraci čekliste, izvršene provere i integrisana promena u repozitorijumu. Premeštanje kartice treba da prati taj stvarni napredak.

Detalj kartice objedinjuje naslov, opis u jednoj rečenici, čeklistu, prioritet, dodeljenu osobu, procenu obima i vezu sa sprintom. Opis i čeklista imaju odvojene uloge: korisnička priča izražava potrebu, dok čeklista pokazuje korake i napredak rada.



Slika 1.2.4 - Forma za kreiranje ili izmenu stavke: naslov, kolona, sprint i opis se definišu pre nego što stavka uđe u aktivan rad.

Board prikazuje samo stavke aktivnog sprinta i njihov status u dogovorenom toku. Premeštanje kartice između kolona treba da prati stvarnu promenu stanja rada, a ne da bude kozmetička radnja pred sastanak.

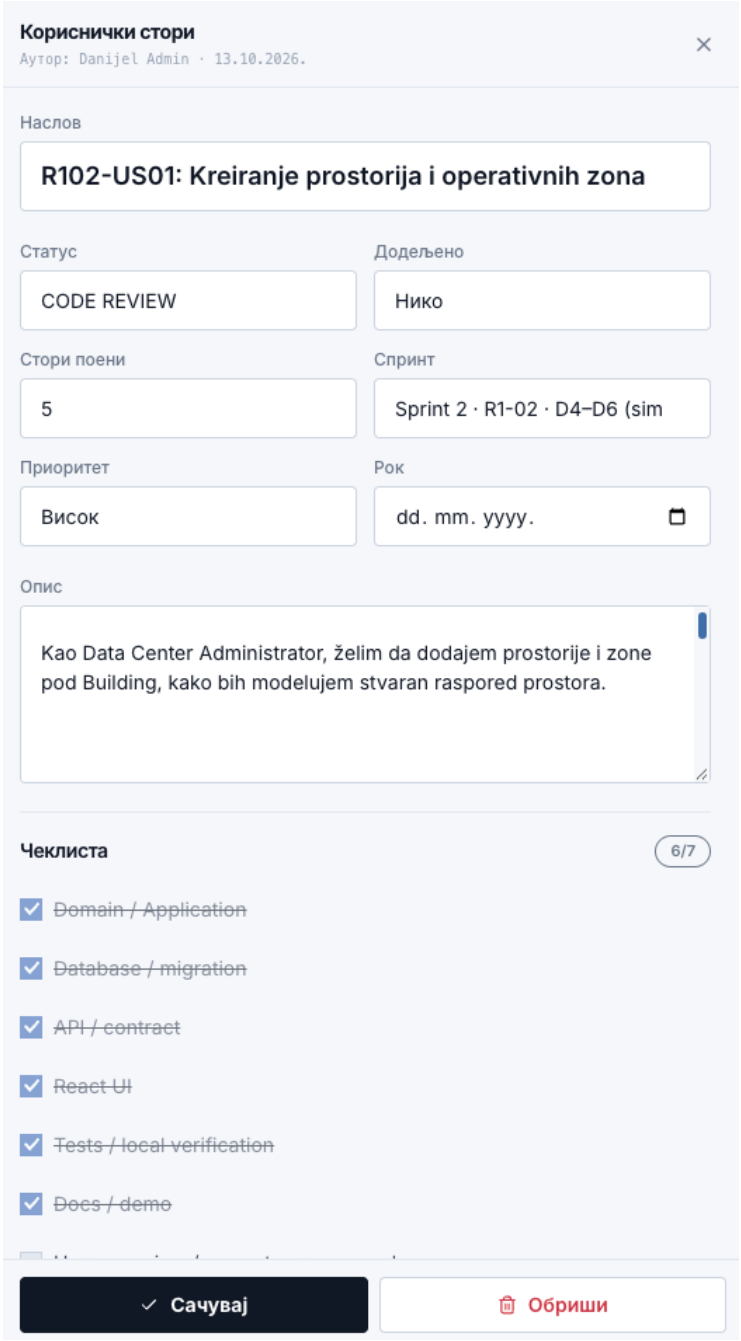
Prioritet i dodeljena osoba pomažu timu da odgovori na pitanje šta raditi sledeće i ko je odgovoran za ishod, dok se aktivnost i komentari čuvaju kao trag odluka i dogovora vezanih za tu stavku - umesto da se izgube u razgovoru van alata.

> **Tapiz Boards u radu tima**
> Korisnička priča u jednoj rečenici, čeklista koraka, status i trag odluka povezuju planiranje sa realizacijom. Sledeći primer iz PYXIS AI prikazuje taj način rada na konkretnim ekranima.

1.2.1. Primer korisničke priče i sprinta - PYXIS AI

Sledeći ekrani prikazuju konkretan primer rada sa korisničkim pričama u Tapiz Boards-u, na projektu „PYXIS AI · R1 · simulirani sprintovi“. Na njima se vidi kako se opis potrebe povezuje sa procenom obima, checklistom, sprintom i statusom rada.

Detalj korisničke priče



Slika 1.2.1.1 - Detalj priče R102-US01: jedna rečenica u opisu i zasebna čeklista koraka.

Priča „R102-US01: Kreiranje prostorija i operativnih zona“ sadrži jednu rečenicu koja povezuje ulogu administratora, dodavanje prostorija i zona i modelovanje stvarnog rasporeda prostora. Isti primer po šablonu:

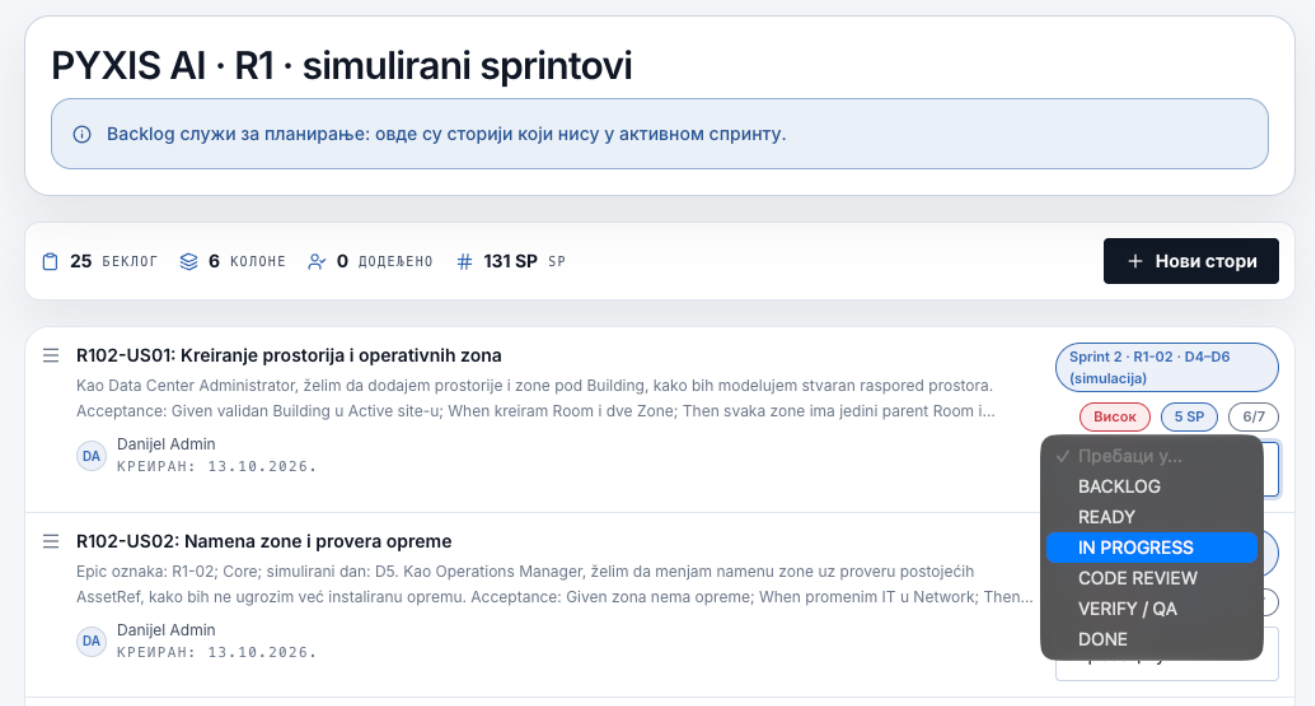
> **Primer po šablonu**
> Kao administrator data centra, želim da dodajem prostorije i zone unutar objekta, kako bih modelovao stvaran raspored prostora.

Naslov, status, dodeljena osoba, stori poeni, sprint, prioritet i rok popunjavaju se u zasebnim poljima. U opisu ostaje samo rečenica po šablonu, a koraci realizacije nalaze se u čeklisti ispod nje.

Procena od 5 SP (Story Points) predstavlja relativnu procenu obima i složenosti rada. Visok prioritet govori o važnosti priče, dok polje „Dodeljeno“ određuje odgovornost za rad. Na ovom snimku priča je u statusu CODE REVIEW, a polje „Dodeljeno“ ima vrednost „Niko“ - tim treba da dogovori odgovornu osobu.

Čeklista prikazuje korake kao što su Domain / Application, Database / migration, API / contract, React UI, Tests / local verification i Docs / demo. Oznaka 6/7 znači da je označeno šest od sedam koraka. Tim označava korak kada je stvarno izvršen i proverava da funkcionalnost ostvaruje radnju i korist opisanu u korisničkoj priči.

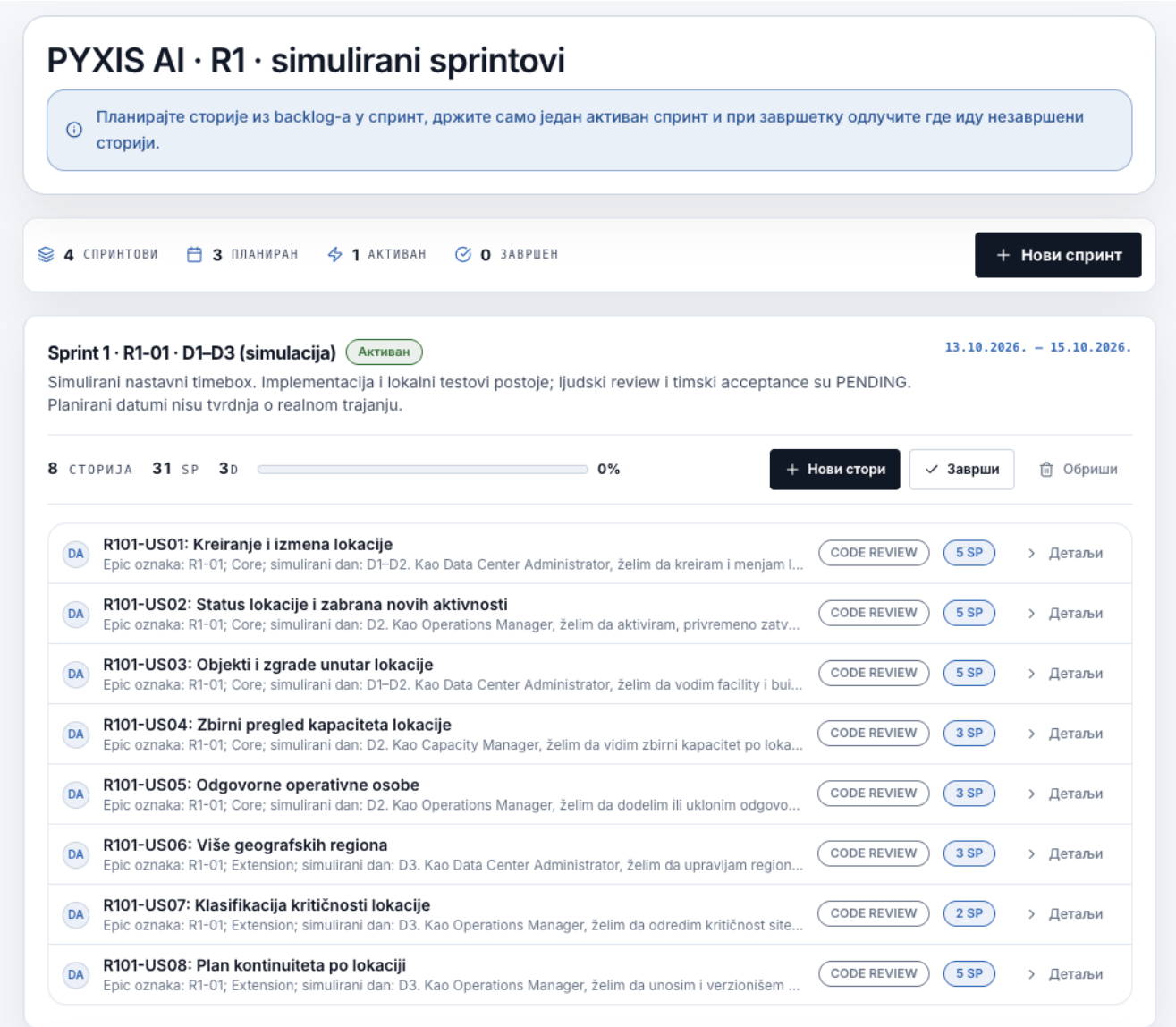
Lista priča i promena statusa



Slika 1.2.1.2 - Lista korisničkih priča i izbor statusa rada.

U listi su vidljivi naslov, sažetak opisa, autor, sprint, prioritet, procena u SP i napredak čekliste. Otvoreni meni prikazuje dogovorene statuse: BACKLOG, READY, IN PROGRESS, CODE REVIEW, VERIFY / QA i DONE. Tim bira status koji odgovara stvarnom stanju rada. Za studentski zadatak opis priče ostaje jedna rečenica po šablonu, kao u prethodnom detaljnom prikazu kartice.

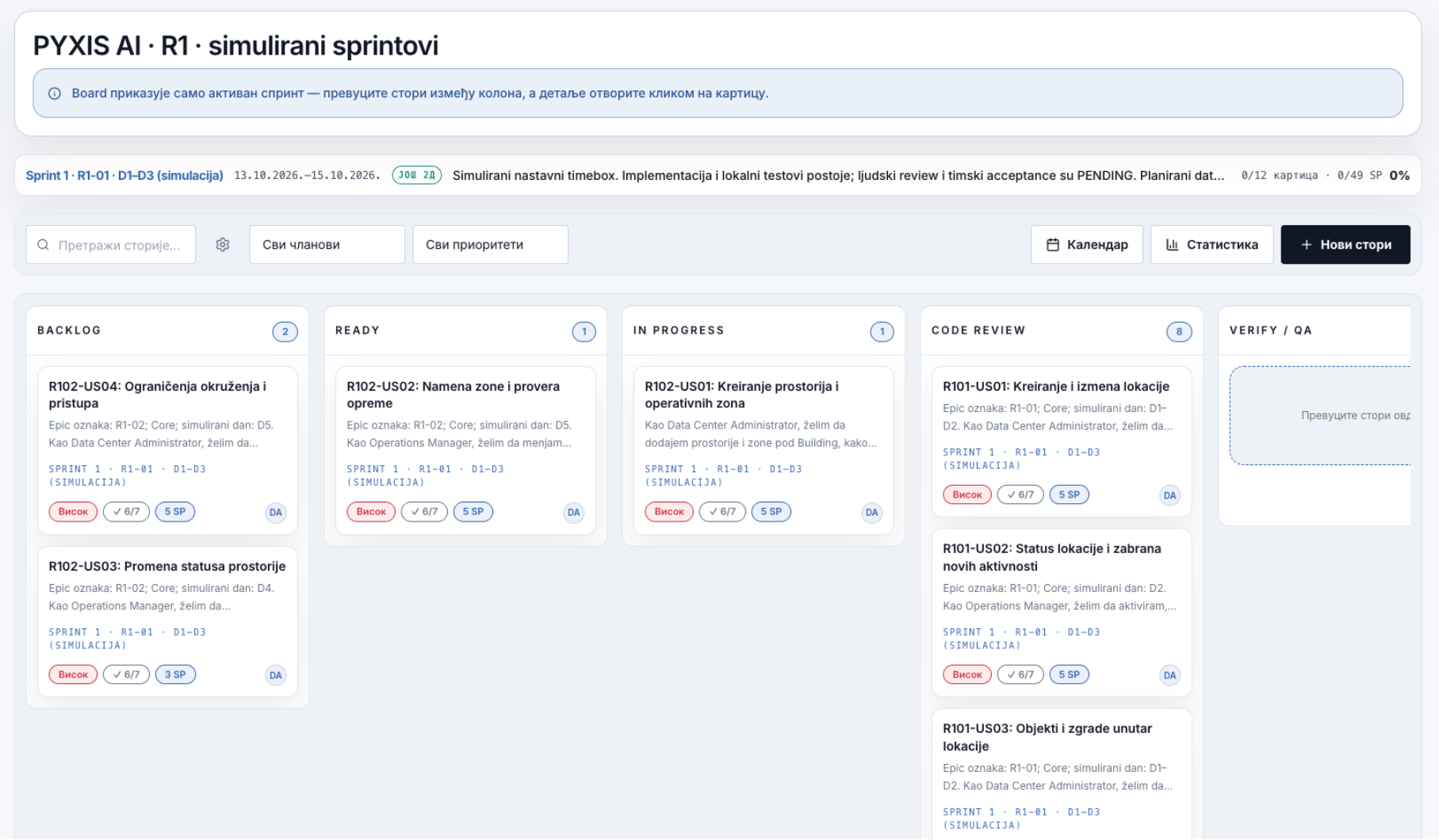
Sprint i njegove korisničke priče



Slika 1.2.1.3 - Aktivni simulirani sprint: osam priča, 31 SP i pregled statusa.

Prikazan je aktivni „Sprint 1 · R1-01 · D1-D3 (simulacija)“, sa osam priča i ukupnom procenom od 31 SP. Pored svake priče nalaze se njen status, procena i pristup detaljima. Sve prikazane priče su u statusu CODE REVIEW, dok je napredak sprinta 0%: implementacija i lokalni testovi mogu da postoje, a da stručni pregled i timsko prihvatanje još nisu završeni. Oznake D1-D3 i datumi služe nastavnoj simulaciji.

Radna tabla aktivnog sprinta



Slika 1.2.1.4 - Radna tabla aktivnog sprinta sa karticama raspoređenim po statusima.

Radna tabla grupiše kartice po statusima BACKLOG, READY, IN PROGRESS, CODE REVIEW i VERIFY / QA; desni deo table nije u celosti obuhvaćen snimkom. Kartica objedinjuje naslov, kratak opis, oznaku sprinta, prioritet, checklistu, procenu i avatar autora. Detalji se otvaraju klikom na karticu, a promena statusa može se obaviti prevlačenjem između kolona. Veći broj kartica u CODE REVIEW ukazuje timu na posao koji čeka pregled.

Snimci predstavljaju različite trenutke nastavne simulacije, pa se broj priča, njihov status i pripadnost sprintu mogu razlikovati između prikaza. Pri poređenju ekrana istu priču prepoznajemo po oznaci, na primer R102-US01.

> **Rad na vežbi - od priče do table**
> Za jednu priču svog projekta uneti naslov i jednu rečenicu po šablonu u polje Opis. Korake realizacije dodati u zasebnu čeklistu. Popuniti polja kartice, planirati priču u sprintu i pokazati kako se njen status menja tokom rada.

1.3. Git kao sledljiv zapis razvoja

Git omogućava da se razvoj rekonstruiše kroz male i smislene promene. Radno stablo sadrži trenutne izmene, staging area određuje šta ulazi u naredni commit, a commit predstavlja imenovanu tačku istorije. U timskom radu posebna grana izdvaja jednu promenu od stabilne glavne grane.

Pojam	Uloga u radu tima
Working tree	Datoteke koje se trenutno menjaju.
Staging area	Izbor sadržaja koji će ući u naredni commit.
Commit	Koherentna promena sa porukom koja opisuje njenu nameru.
Branch	Izdvojena linija razvoja za jednu funkcionalnost, ispravku ili drugi ograničen posao.
Remote	Udaljeni repozitorijum preko kog tim razmenjuje promene.

Tabela 1.3.1

Listing 1.3.1 - BASH

Primer malog i proverljivog razvojnog koraka

```
git status
git diff
git add src/Application/ReservationService.cs
git add tests/ReservationServiceTests.cs
git diff --staged
git commit -m "Add reservation overlap rule"
git push -u origin feature/reservation-overlap
```

- Pre commit-a pregledati pripremljeni diff i razumeti svaku uključenu promenu.
- Jedan commit treba da predstavlja jednu koherentnu nameru kad god je to praktično moguće.
- Promene koje nisu povezane sa ciljem stavke ne treba usput uključivati u isti pull request.
- Glavna grana treba da ostane stabilna i spremna za zajednički rad tima.

1.4. Pull request, pregled koda i konflikti

Pull request je mesto na kome se zahtev povezuje sa konkretnom implementacijom. Pregledalac proverava da li je promena u odgovarajućem obimu, da li poštuje arhitektonska pravila, da li postoje potrebni testovi i da li se iz diff-a može razumeti namera autora. Tek nakon pregleda i provere promena se integriše u glavnu granu.

Tok jedne promene

01	**Stavka** korisnička priča i čeklista
02	**Grana** ograničena implementacija
03	**Commit-i** smisleni razvojni koraci
04	**Pregled** kod, testovi i rizici
05	**QA i merge** provereno ponašanje i integracija

Slika 1.4.1 - Pull request povezuje poslovni zahtev sa implementacijom i dokazima provere.

Konflikt pri spajanju nastaje kada Git ne može samostalno da odredi konačni sadržaj. Rešavanje konflikta nije uklanjanje tehničkih markera, već odluka o tome koje ponašanje treba da ostane nakon spajanja paralelnih promena.

Listing 1.4.1 - BASH

Rešavanje konflikta uz završnu proveru

```
git checkout main
git pull
git checkout feature/reservation-overlap
git merge main
# rešiti konflikt i pregledati konačan diff
git add src/Application/ReservationService.cs
git commit -m "Resolve reservation rule conflict"
dotnet test
```

Prepisivanje zajedničke istorije
`git push --force` nije deo uobičajenog timskog toka. Na deljenim granama može ukloniti tuđe commit-e i koristi se samo kada tim razume posledice i postoji opravdan razlog.

Ishod vežbe
Student ume da poveže zahtev, backlog, Git istoriju, pull request i proveru ponašanja u jedan sledljiv razvojni proces.